

Base de données NoSQL

Fragmentation

Louis Roussel

louis.roussel@univ-lille.fr

Support de cours par **Bastien Cazaux** qui s'appuie sur les cours d'**Anne Etien** et de **Anne Cécile Caron** de l'université de Lille et celui de **Régis Behmo** et **Nicolas Travers** de CentralSupélec.

Plan du cours

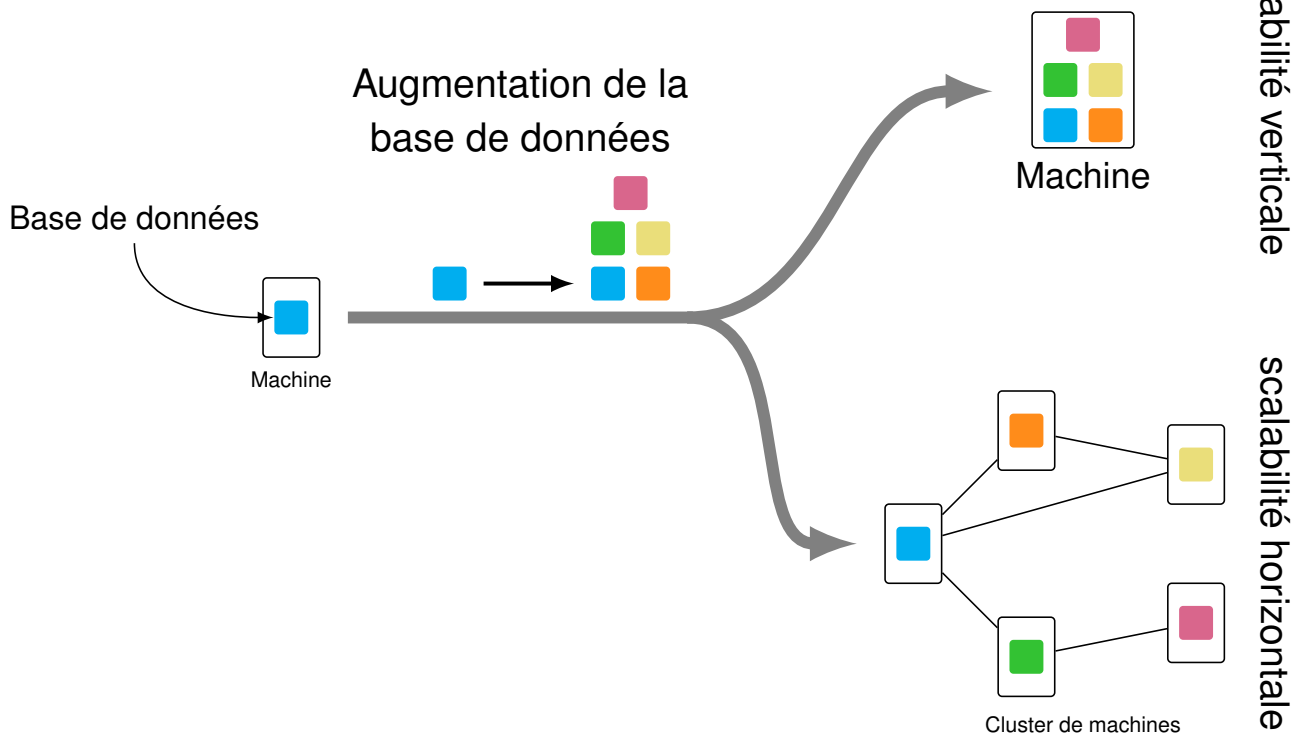
Plan du cours		
Introduction	3	
Un seul serveur		Table de hachage classique
Scalabilité horizontale et verticale		Hachage linéaire
Scalabilité horizontale et verticale : schéma		Hachage linéaire : exemple
Scalabilité horizontale : Fragmentation des données		Hachage linéaire : exercice
Fragmentation par agrégat	7	Hachage linéaire distribué
Exemple de fragmentations horizontale et verticale		Consistent Hashing
Fragmentation horizontale : optimiser les requêtes		Consistent Hashing : exercice
Distribution	9	Consistent Hashing : Amélioration
		MapReduce
		MapReduce
		Architecture Distribuée : MapReduce
		MapReduce : exercice
		Fragmentation : problèmes

Un seul serveur

- Pas de distribution
- Toutes les données sont sur la même machine qui gère lecture et écriture
- Solution simple pour gérer la BD et concevoir l'application
- Fait du sens même pour du NoSQL
 - En particulier pour BD orientée graphe
- **A utiliser si on peut en priorité**

Scalabilité horizontale et verticale

- Explosion de la taille de certaines données à gérer
 - les bases de données traditionnelles n'arrivent pas à fournir un temps de réponse satisfaisant
- Deux solutions envisageables :
 - **Scalabilité verticale** : Augmenter les ressources de la machine
 - la puissance du processeur, la mémoire, ...**Inconvénients**
 - on ne peut pas augmenter la puissance du processeur à l'infini
 - l'augmentation des performances des composants est suivie d'une augmentation du prix encore plus importante
 - **Scalabilité horizontale** : Utiliser un cluster de machines (ensemble de machines)
Avantages
 - permet une augmentation illimitée des capacités avec un coût de revient moindre (suffit d'ajouter d'autres machines au cluster)



Scalabilité horizontale : Fragmentation des données

- **Objectifs :**
 - Différentes parties des données sur différents serveurs
 - Chaque serveur gère ses propres lectures et écritures
 - La charge entre les serveurs est répartie de façon pertinente.

Exemple : Si on a 10 serveurs, chaque serveur doit gérer seulement 10% de la charge.
- Dans l'idéal chaque utilisateur parle à un seul serveur pour avoir des réponses rapides de ce serveur.
- **Problématique :**
 - Comment organiser les données pour que chaque utilisateur récupère la plupart de ses données d'un seul serveur ?
- **Solutions :**
 - Mettre les agrégats utilisés ensemble sur le même serveur
 - Les agrégats = rassemblement de données utilisées ensemble
 - Utiliser une méthode pour distribuer les données au fur et à mesure

Exemple : Distributed Linear Hashing (LH*) ou Consistent Hashing

Exemple de fragmentations horizontale et verticale

Original Table

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
1	TAEKO	OHNUKI	BLUE
2	O.V.	WRIGHT	GREEN
3	SELDA	BAĞCAN	PURPLE
4	JIM	PEPPER	AUBERGINE

Vertical Partitions

VP1

CUSTOMER ID	FIRST NAME	LAST NAME
1	TAEKO	OHNUKI
2	O.V.	WRIGHT
3	SELDA	BAĞCAN
4	JIM	PEPPER

VP2

CUSTOMER ID	FAVORITE COLOR
1	BLUE
2	GREEN
3	PURPLE
4	AUBERGINE

Horizontal Partitions

HP1

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
1	TAEKO	OHNUKI	BLUE
2	O.V.	WRIGHT	GREEN

HP2

CUSTOMER ID	FIRST NAME	LAST NAME	FAVORITE COLOR
3	SELDA	BAĞCAN	PURPLE
4	JIM	PEPPER	AUBERGINE

Fragmentation horizontale : optimiser les requêtes

PRODUCT	PRICE
WIDGET	\$118
GIZMO	\$88
TRINKET	\$37
THINGAMAJIG	\$18
DOODAD	\$60
TCHOTCHKE	\$999



(\$0-\$49.99)

PRODUCT	PRICE
TRINKET	\$37
THINGAMAJIG	\$18

(\$50-\$99.99)

PRODUCT	PRICE
GIZMO	\$88
DOODAD	\$60

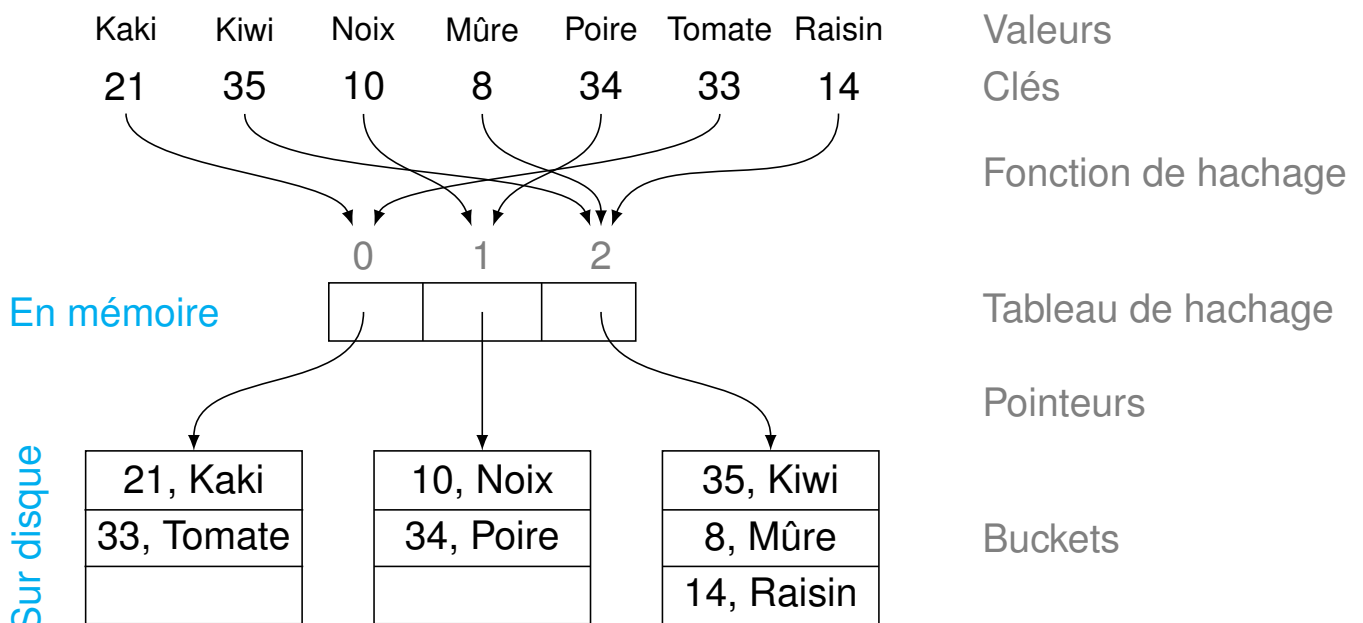
(\$100+)

PRODUCT	PRICE
WIDGET	\$118
TCHOTCHKE	\$999

Table de hachage classique

On a un ensemble de paires (k, v) clé-valeurs :

- Une fonction de hachage distribue les valeurs (v) dans M alvéoles (buckets) en fonction de la clé (k) .
- Chaque alvéole contient un nombre maximum de valeurs.
- On suppose qu'on lit une alvéole sur le disque en temps constant.



Hachage linéaire

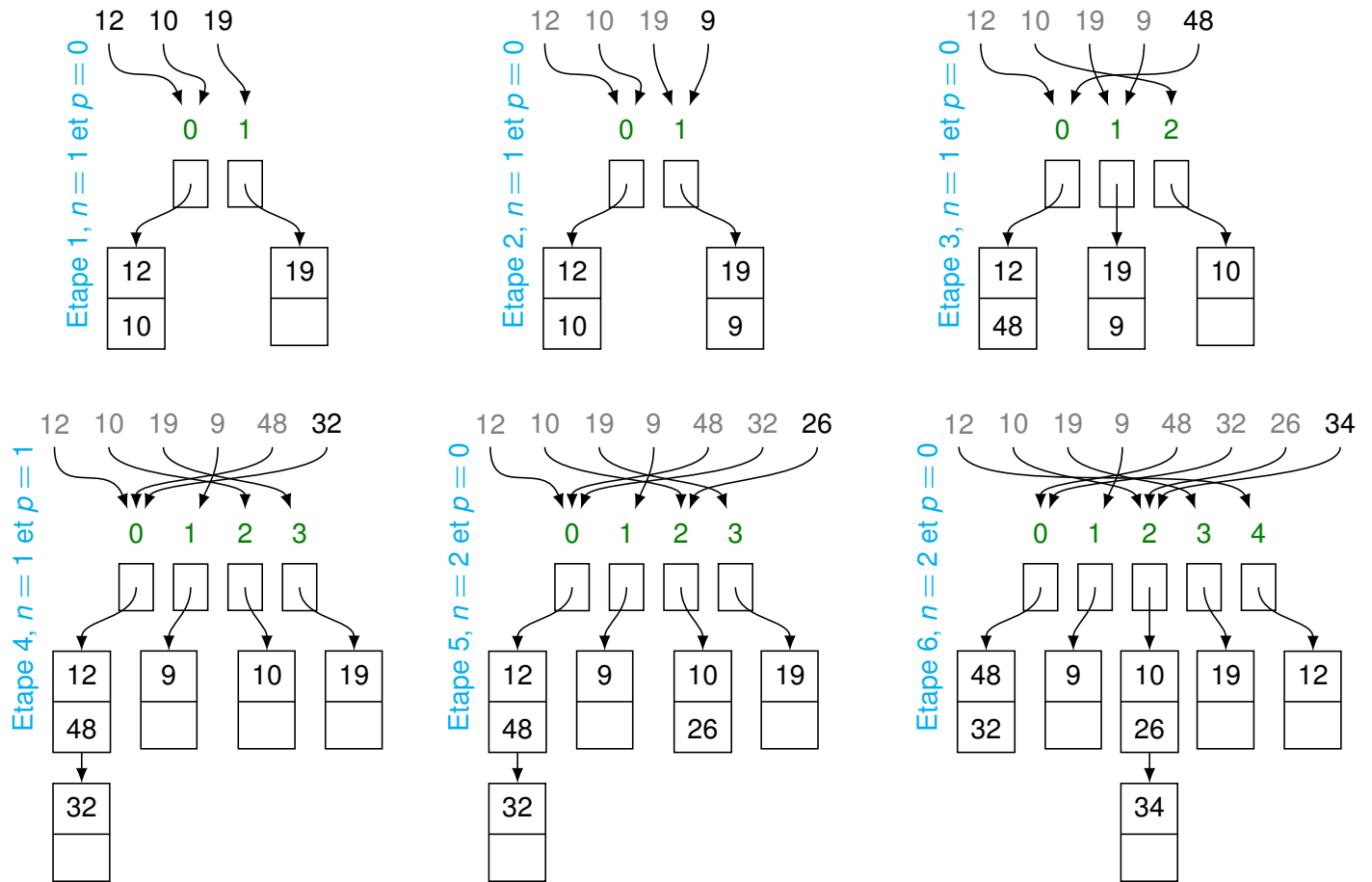
- Quand une alvéole b est pleine :
 - on chaîne une nouvelle alvéole à b , sans changer M
 - on divise une alvéole b_p , souvent distincte de b . p est un indice incrémenté à chaque division. M est augmenté de 1.
- Initialement, $p = 0$, l'alvéole b_0 est donc la première qui doit se diviser.
- On utilise 2 fonctions de hachage pour le niveau n (t.q. $2^n \leq M < 2^{n+1}$) :
 1. h_n pour les alvéoles b_i t.q. $i \in [p, M - p - 1]$
 2. h_{n+1} pour les autres alvéoles

où $h_t : k \mapsto (k \bmod 2^t)$
- Retrouver l'alvéole de la clé k :


```

      i ← hn(k)
      if (i < p) i ← hn+1(k)
      return addr(bi);
      
```

Hachage linéaire : exemple



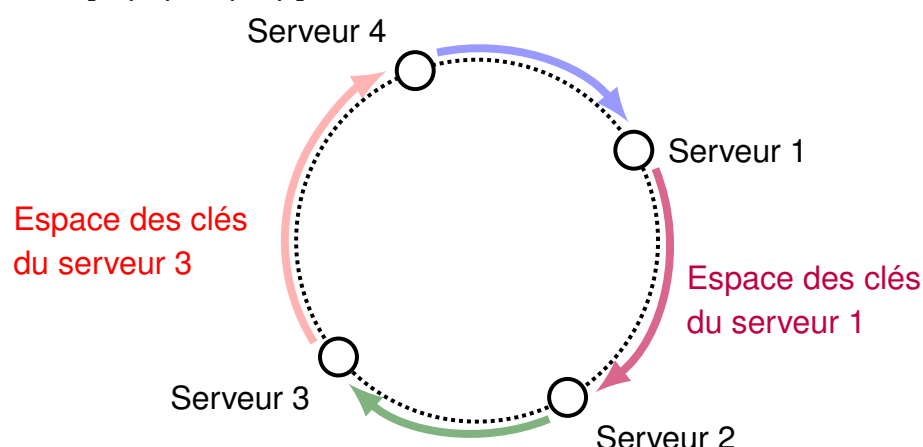
Hachage linéaire : exercice

Quel est l'état final d'un hachage linéaire où on a une taille d'alvéole de 2 et un ensemble de clés 3, 6, 12, 32, 33, 54, 7, 21, 34, 11 (ajouté dans cet ordre) ?

- LH^* (Distributed Linear Hashing) est l'algorithme de hachage linéaire où les alvéoles sont des serveurs.
- Les noeuds doivent avoir connaissance :
 - du niveau n qui détermine le couple (h_n, h_{n+1}) à utiliser
 - de la valeur du pointeur p
 - des changements des adresses des serveurs
- Problème de cohérence :
 - Soit tous les participants ont des informations parfaitement à jour ; les recherches sont rapides, mais les insertions avec modification de structure sont lentes.
 - Soit on autorise que les participants n'aient pas tous la même information ; les recherches sont un peu plus ralenties mais le coût de maintenance de la structure est plus faibles. C'est ce compromis qu'encourage l'algorithme. En effet, en utilisant un algorithme de redirection, on peut retrouver le bon serveur en quelques sauts.
- L'algorithme permet d'ajouter dynamiquement des serveurs quand le nombre de clés augmente. Mais on suppose que tous les serveurs sont stables.

Consistent Hashing

- l'ajout ou la suppression de serveurs ne change pas fondamentalement l'affection clé $\rightarrow$ serveur.
- la fonction de hachage h envoie à la fois les clés et les adresses IP des serveurs sur un très grand espace d'adressage A , par exemple $[0, 2^{64} - 1]$.
- A est organisé en anneau, parcouru dans le sens des aiguilles d'une montre.
- Si S et S' sont deux serveurs adjacents sur l'anneau, toutes les clés dans $[h(S), h(S')]$ sont envoyées sur S .



Consistent Hashing : exercice

Quel est l'état final d'un Consistent Hashing où on a un nombre de serveur de 3 et un ensemble de clés 3, 6, 12, 32, 33, 54, 7, 21, 34, 11 (ajouté dans cet ordre) ?

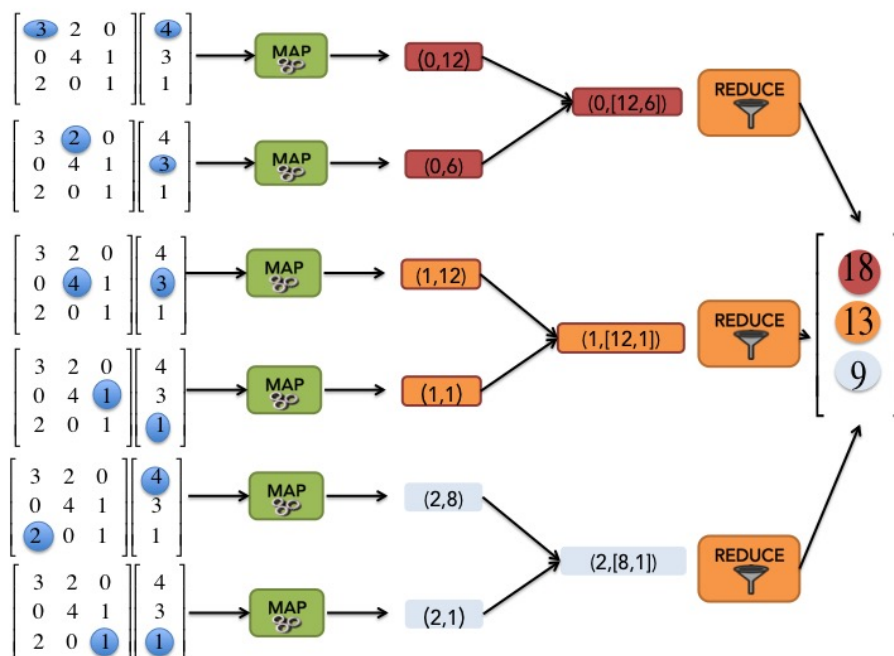
Consistent Hashing : Amélioration

- On aimerait que les serveurs soient bien répartis sur l'anneau ; pour que les serveurs soient chacun responsables potentiellement du même volume de données.
On va définir des noeuds virtuels : l'anneau contient un grand nombre de noeuds virtuels et on va répartir ces noeuds virtuels sur les machines physiques. Chaque machine physique reçoit beaucoup de noeuds virtuelles donc il y a de grandes chances que ça s'équilibre.
- On aimerait avoir la tolérance aux pannes. On va répliquer les données n fois (voir cours sur la réplication).

- **MapReduce** est un patron d'architecture de développement informatique dans lequel sont effectués des calculs parallèles et distribués, de données potentiellement très volumineuse.
- Plutôt que d'envoyer les données vers le serveur qui exécute un programme, on déplace le programme vers les données
- Principe de localité : les données d'une machines sont lues par le programme exécuté sur cette même machine.
- Comme le nom l'indique, le programme est basé sur 2 fonctions : la fonction **map** qui filtre les données initiale et le fonction **reduce** qui construit le résultat par agrégation.

Architecture Distribuée : MapReduce

- Fonctionnement :
 1. Séparer les données en blocs
 2. Une fonction **Map** qui s'applique à chaque bloc
 3. Une fonction **Reduce** qui fusionne l'ensemble des résultats de chaque bloc



- Faire une fonction *MapReduce* pour résoudre le problème suivant.
- On a un texte $\{XBB, CBA, XAC\}$ et on veut connaître le nombre d'occurrence de chaque lettre dans le texte ($\{A : 2, B : 3, C : 2, X : 2\}$).

Fragmentation : problèmes

- Exemple les clients de A à D sur un serveur et ceux de E à G sur un autre
- Résilience : si l'un des serveurs tombe les données sur ce serveur ne sont plus accessibles
 - Mais seulement celles-là
- Compliqué de migrer vers un mode *fragmentation*
 - Commencer avec dès la conception
 - Passer du serveur seul au mode *fragmentation* avant qu'il ne soit trop tard
 - Ne pas passer au mode *fragmentation* car les données et les applications ne sont pas adaptées.